**ChatGPT said:**

# BST Insertion and Dynamic Balancing

## 1. Binary Search Tree (BST)

A Binary Search Tree is a special type of binary tree in which each node contains a value and follows a specific ordering property. The values smaller than the root node are stored in the left subtree, while the values greater than the root node are stored in the right subtree. Each node can have at most two children.

**Properties of BST:**

- The left subtree contains smaller values than the root.
- The right subtree contains greater values than the root.
- The left and right subtrees also follow the BST property.
- Duplicate values are handled according to the implementation.

## 2. BST Insertion

Insertion is the process of adding a new node to a Binary Search Tree at its correct position without violating the BST property.

**Algorithm of BST Insertion:**

1. If the tree is empty, the new node becomes the root.
2. Compare the new value with the root node.
3. If the new value is smaller, move to the left subtree.
4. If the new value is greater, move to the right subtree.
5. Repeat the comparison until an empty position is found.
6. Insert the new node at that position.

**Example:**

Consider the insertion of elements 50, 30, 70, 20 and 40.

Plain text

```
    50
   /  \
  30   70
 / \
20  40
```

In this example, 30 is inserted to the left of 50 because it is smaller, while 70 is inserted to the right because it is greater. Similarly, 20 and 40 are inserted according to the BST property.

## 3. Dynamic Balancing

Dynamic balancing is the process of maintaining the height of a binary search tree after insertion or deletion of nodes. It prevents the tree from becoming highly skewed and helps maintain efficient searching, insertion and deletion operations.

When nodes are inserted in sorted order, an ordinary BST may become skewed, resulting in poor performance. Dynamic balancing techniques restructure the tree whenever necessary to maintain its balance.

**Example of an unbalanced BST:**

Plain text

```
10
  \
   20
     \
      30
        \
         40
```

**Balanced form:**

Plain text

```
    20
   /  \
  10   30
         \
          40
```

The balanced tree has a smaller height and can perform operations more efficiently.

## 4. Need for Dynamic Balancing

Dynamic balancing is required to maintain the efficiency of a Binary Search Tree. An unbalanced tree may behave like a linked list, increasing the time required to search, insert or delete an element. Balancing reduces the tree height and helps maintain logarithmic time complexity for these operations.

## 5. Techniques of Dynamic Balancing

**AVL Tree**

An AVL tree is a self-balancing Binary Search Tree in which the height difference between the left and right subtrees of every node is at most one. This difference is called the **balance factor**.

$$\text{Balance Factor} = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$$

The balance factor of each node in an AVL tree must be −1, 0 or +1. If insertion or deletion violates this condition, rotations are performed to restore balance.

#### Red-Black Tree

A Red-Black Tree is a self-balancing Binary Search Tree in which every node is assigned either a red or black color. Specific coloring rules ensure that the tree remains approximately balanced. Recoloring and rotations are used to restore these rules after insertion or deletion.

#### Tree Rotations

Rotations are structural operations used to restore balance in a binary search tree while preserving the BST ordering property.

The four main types of rotations are:

1. **Left Rotation:** Used to correct certain right-heavy configurations.
2. **Right Rotation:** Used to correct certain left-heavy configurations.
3. **Left-Right Rotation:** A left rotation followed by a right rotation.
4. **Right-Left Rotation:** A right rotation followed by a left rotation.

## 6. Time Complexity

| Operation | Ordinary BST (Worst Case) | Balanced BST |
|---|---|---|
| Searching | $O(n)$ | $O(\log n)$ |
| Insertion | $O(n)$ | $O(\log n)$ |
| Deletion | $O(n)$ | $O(\log n)$ |

Here, $n$ represents the number of nodes in the tree.

## Conclusion

BST insertion places a new node at its appropriate position according to the Binary Search Tree property. Dynamic balancing maintains a suitable tree height after insertion and deletion. Techniques such as AVL trees, Red-Black trees and tree rotations help improve the efficiency of tree operations.